CS 422 Assignment 4: Maximum Concurrent Flow and Network Design

Kush, Joseph, Issac

Repository: `https://github.com/krash3125/cs422-assignment-4`

1 Part 1: Uniform Traffic Matrix

Main idea

Every node sends $\frac{d}{n-1}$ to each of the other $n-1$ nodes, so each node sends d in total and the whole network has to carry nd. The network also has exactly nd capacity (n nodes, d links each, capacity 1). A flow that takes 1 hop uses 1 unit of capacity per unit of traffic, a flow that takes 2 hops uses 2, and so on. This gives

$$\theta \cdot \frac{d}{n-1} \sum_{s \neq t} \text{dist}(s,t) \leq nd \quad \Longrightarrow \quad \theta \leq \frac{1}{\text{average shortest path length}}$$

So a good topology needs two things:

1. **Short paths:** the average number of hops should be as small as possible.
2. **Balanced load:** the bound above is only reached if every link is fully used. If some links are busier than others, the busiest link limits θ before the rest are full.

How short can paths be? (upper bound for any topology)

From any node, with d outgoing links we can reach at most d nodes in 1 hop, at most d^2 in 2 hops, d^3 in 3 hops, and so on (the Moore bound). Let $M(n,d)$ be the hop total we get by filling these levels in order until all $n-1$ other nodes are placed. No node can do better, so for *every* topology

$$\theta \leq \frac{n-1}{M(n,d)}.$$

For example, with $n = 9$, $d = 2$: 2 nodes at 1 hop, 4 at 2 hops, 2 at 3 hops, so $M = 2 + 8 + 6 = 16$ and $\theta \leq 8/16 = 0.5$.

Our topology

To get close to this tree-like spreading, we think of each node as a number in base d. Each node connects to the nodes we get by shifting its digits left and adding a new last digit. Every hop adds one digit, so any node can be reached in about $\log_d n$ hops. Written with mod n so it works for any n, this is the Imase–Itoh graph [1]:

1. For every node $i = 0, \ldots, n-1$ and every $j = 1, \ldots, d$, add a link $i \to (-d \cdot i - j) \bmod n$.

2. If some node a has a link to itself, pick another link $u \to v$ (not touching a, and where $u \to a$ and $a \to v$ do not already exist) and replace $a \to a$ and $u \to v$ with $u \to a$ and $a \to v$. Every node still has d links in and d links out.

Why this is a valid topology:

- Every node has d links out by construction. It also has d links in: since $d \mid n-1$, d and n share no factor, so $i \mapsto -d \cdot i \bmod n$ is a one-to-one map. For any node v and each j there is exactly one i with $-d \cdot i - j \equiv v$.
- Node i links to itself only when $(d+1)\, i \equiv -j \pmod{n}$, which holds for at most one j, so each node has at most one self-loop. Step 2 removes them (a self-loop wastes a link).
- Before step 2, the Imase–Itoh graph has diameter at most $\lceil \log_d n \rceil$ [1], which is as small as the Moore bound allows (up to rounding).
- When $n = d+1$, $-d \equiv 1 \pmod{n}$, so node i links to $i-1, \ldots, i-d$, which is every other node. This is the complete graph, every flow takes 1 hop, and $\theta = 1$ is optimal.

How close to optimal is it?

We checked our topology with `part1.py`:

- **Short paths:** for every $n < 120$ with $d \mid n-1$, step 2 always succeeds, every node has d links in and out, and the average path length is at most 14.1% above the Moore bound (worst case $n = 15$, $d = 2$).
- **Balanced load:** short paths alone are not enough. Our graph is not symmetric, so some links carry more traffic than others. We solved the exact maximum concurrent flow (a linear program) on our topology:

n	d	avg. path length	θ of our topology	upper bound (any topology)	ratio
9	2	2.167	0.364	0.500	72.7%
13	3	1.955	0.500	0.571	87.5%
16	3	2.167	0.417	0.500	83.3%
17	4	1.835	0.500	0.571	87.5%
21	4	1.993	0.497	0.556	89.5%
25	3	2.418	0.405	0.421	96.1%

So our topology is **near-optimal, not provably optimal**. The ratio is measured against the upper bound, so the true gap to the best topology can only be smaller. For example, for $n = 9$, $d = 2$ the Part 2 integer program (run with the uniform matrix) finds that the best possible topology gets 0.488, compared with our 0.364.

Why we do not claim exact optimality. The upper bound cannot always be reached. When $n = 1 + d + \cdots + d^k$ (for example $n = 7$, $d = 2$), reaching it would need a directed Moore graph, and these do not exist for $d \geq 2$, $k \geq 2$ [2]. In those cases the true optimum has no known formula (for $n = 7$, $d = 2$ the integer program finds 6/11). For other n the bound can be reached exactly (Part 2's uniform case, $n = 8$, $d = 4$, gets 0.7), but the best topologies the solver finds for small n are irregular graphs with no pattern we could generalize. Finding the exactly optimal topology for every n and d is an open problem. The usual approach in the literature is the one we use: a low-diameter graph compared against the upper bound [3, 4].

Example: $n = 9$, $d = 2$

Node i links to $(-2i - 1) \bmod 9$ and $(-2i - 2) \bmod 9$ (no self-loops occur here):

node	links to
0	8, 7
1	6, 5
2	4, 3
3	2, 1
4	0, 8
5	7, 6
6	5, 4
7	3, 2
8	1, 0

The average path length is 2.17 against the best possible 2.0, so this graph can reach at most $1/2.17 \approx 0.46$. Because the load is not perfectly balanced, the exact maximum concurrent flow is 0.364. The upper bound for any topology is 0.5, and the best topology found by the solver gets 0.488.

Implementation

Code: `https://github.com/krash3125/cs422-assignment-4/blob/main/part1.py`

- `build_topology(n, d)`: steps 1 and 2 above.
- `path_length_sum` and `moore_sum`: average path length of our graph and the Moore bound.
- `concurrent_flow`: exact θ on a fixed topology (Gurobi linear program).

2 Part 2: Arbitrary Traffic Matrix

Main idea

In Part 1 every pair sent the same amount, so the best we could do was make every node close to every other node. Here the traffic matrix T is given, and some pairs send a lot more than others. So the topology should depend on T: pairs with a lot of traffic should get direct links (even more than one link), and pairs with little traffic can share capacity over 2 or more hops.

Choosing the links and routing the flow depend on each other, so we put both into one integer program. For each given T, we solve it, and the solution gives the links to build. Since the solver looks at every possible topology and routing and returns the best one, the topology it gives is the one that maximizes concurrent flow for that T. Here $n = 8$ and $d = 4$.

Formulation

Decision variables.

- $x_{uv} \in \{0, 1, \ldots, 4\}$ (integer), for $u \neq v$: number of links from u to v. More than one link between the same pair is allowed.
- $f^s_{uv} \geq 0$: amount of flow from source s that goes over link $u \to v$.
- $\theta \geq 0$: the fraction of every demand that gets delivered.

Objective.

$$\max\ \theta$$

Constraints.

1. **Degree:** every node has 4 links out and 4 links in.

$$\sum_{v \neq u} x_{uv} = 4, \qquad \sum_{v \neq u} x_{vu} = 4 \qquad \forall u$$

2. **Flow conservation:** source s sends $\theta \cdot T_{st}$ to every destination t.

$$\sum_{v} f^s_{sv} - \sum_{v} f^s_{vs} = \theta \sum_{t} T_{st} \qquad \forall s$$

$$\sum_{v} f^s_{vw} - \sum_{v} f^s_{wv} = \theta\, T_{sw} \qquad \forall s,\ \forall w \neq s$$

3. **Capacity:** each link has capacity 1, so the total flow on $u \to v$ is at most the number of links there.

$$\sum_{s} f^s_{uv} \leq x_{uv} \qquad \forall u \neq v$$

We group flow by source instead of by (source, destination) pair. The flow from s only needs to drop off θT_{st} at each t, so this is enough and uses fewer variables.

Why this works

- The degree constraints make sure x is a valid topology (4 links in and out per node).
- The flow and capacity constraints make sure the traffic θT can actually be routed on that topology.
- The solver maximizes θ over all valid topologies and routings together, so the answer is the best topology for this T. It adapts to T because T shows up in the flow conservation constraints, so a different T gives different links.
- The intuition shows up in the results: big entries of T get direct links, and if T is something like $4\times$ a permutation, the solver just builds those links and gets $\theta = 1$.
- If $T = 0$ (allowed by the hose model) there is nothing to route and θ is unbounded; the code reports this instead of returning a number.

Implementation (Gurobi)

Code: `https://github.com/krash3125/cs422-assignment-4/blob/main/topology.py`

- `best_topology(T)`: builds and solves the integer program and returns θ and the links.
- Variables x, f, θ; objective; degree constraints; flow conservation; capacity.
- `random_hose_tm()`: makes a random hose traffic matrix (a random mix of permutations, scaled by 4) for testing.

Results

traffic matrix	best θ
uniform ($\frac{4}{7}$ to every pair)	0.700
random hose #0	0.875
random hose #1	0.861
random hose #2	0.827

For the uniform matrix the solver gets 0.700. This matches the Part 1 upper bound: with $d = 4$, each node can reach 4 nodes in 1 hop and the other 3 in 2 hops, so the average path length is at least $10/7$ and $\theta \leq 7/10$. Here the bound is reached exactly. The random matrices get higher θ because their traffic is concentrated on fewer pairs, so more of it can go over direct links.

References

[1] M. Imase and M. Itoh. A design for directed graphs with minimum diameter. *IEEE Transactions on Computers*, C-32(8):782–784, 1983.

[2] W. G. Bridges and S. Toueg. On the impossibility of directed Moore graphs. *Journal of Combinatorial Theory, Series B*, 29(3):339–341, 1980.

[3] A. Singla, P. B. Godfrey, and A. Kolla. High throughput data center topology design. In *NSDI*, 2014.

[4] V. Addanki, C. Avin, and S. Schmid. Mars: Near-optimal throughput with shallow buffers in reconfigurable datacenter networks. In *SIGMETRICS*, 2023.